

WildlifeSystems - biodiversity technologies

Ed Baker

2026-01-22

Contents

About	7
Support	7
1 Biodiversity Technologies	9
1.1 What are Biodiversity Technologies?	9
1.2 Overall Philosophy	10
1.3 Structure of Wildlife Systems	11
2 Nodes	13
2.1 Standard node	13
2.2 Enclosures	14
3 How nodes work	15
3.1 Audio	15
4 Sensor Networks	17
5 Environmental Sensors	19
5.1 How sensors work	19
6 Sensors in WildlifeSystems	21
6.1 Devices included in the base system	21
6.2 Installing sensor support	22
6.3 Reading data from a device	22
6.4 The sensor reading process	22
6.5 Installing new sensors	23

7 Supported sensors	25
7.1 List of supported sensors	25
8 Implementing new sensors	27
8.1 Reading the sensor	27
8.2 Submitting packages to WildlifeSystems	28
9 Sensor Data Tools	29
9.1 Installation	29
9.2 sensor-data	29
9.3 sensor-mon	34
9.4 sensor-plot	35
9.5 Data Formats	37
10 Sound Devices	39
10.1 How sound devices work	39
11 Sound devices in WildlifeSystems	41
11.1 Installing sound device support	41
11.2 Supported sound devices	41
11.3 Installing a sound device	42
12 Imaging Devices	43
12.1 How imaging devices work	43
13 Imaging devices in WildlifeSystems	45
14 Power Management	47
14.1 Installation of power management tools	47
14.2 Turning functionality on and off	47
14.3 Considerations	47
15 Indicators and heartbeats	49
15.1 Installation of <code>ws-indcate</code> and <code>ws-heartbeat</code>	49
15.2 Indicators	49
15.3 Heartbeat	49

<i>CONTENTS</i>	5
16 Integration with monitoring tools	51
16.1 PRTG	51
17 Developer Guidelines	53
17.1 Documentation	53
17.2 File system layout	53
18 Server Tools	55
18.1 Installation	55
18.2 Adding nodes and receieving a token	55
18.3 Removing a node	56
A A note on sudo	57
B Return codes	59
B.1 00-09 Script functionality	59
B.2 10-19 Parameter problems	59
B.3 20-29 Sensor problems	59
B.4 30-39 Sound device problems	60
B.5 40-49 Image device problems	60
B.6 50-59 Power management problems	60
B.7 60-69 Special meanings	60

About

This book explains the technologies developed as part of WildlifeSystems and how they can be implemented in real-world scenarios.

Support

WildlifeSystems nodes were originally developed and used as part of the Leverhulme Trust funded Automated Acoustic Observatories project at the University of York. Additional development is undertaken as part of the Urban Nature Project at the Natural History Museum, London.

Chapter 1

Biodiversity Technologies

1.1 What are Biodiversity Technologies?

Biodiversity technologies are tools that allow researchers and others to study, monitor and conserve biodiversity. These tools can be used to monitor a wide range of species, habitats, and ecosystems, potentially in (near) real-time and/or from a great distance away.

1.1.1 What has enabled Biodiversity Technologies?

The development of biodiversity technologies has been enabled by the rapid advances in sensor technology, data processing, and communication networks. These technologies have made it possible to collect, store, and analyse large amounts of data from a wide range of sources, including remote sensors, cameras, and acoustic recorders.

This has been accompanied by a decline in the cost of these technologies (e.g. Figure 1.1), making them more accessible to researchers, conservationists, and others interested in monitoring biodiversity.

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): NAs introduced by coercion
```

These technologies can potentially create vast datasets, and in order to adhere to the FAIR principles (Findable, Accessible, Interoperable, Reusable), it is important for the data to be accompanied by an appropriate suite of data standards, such as those developed by TDWG.

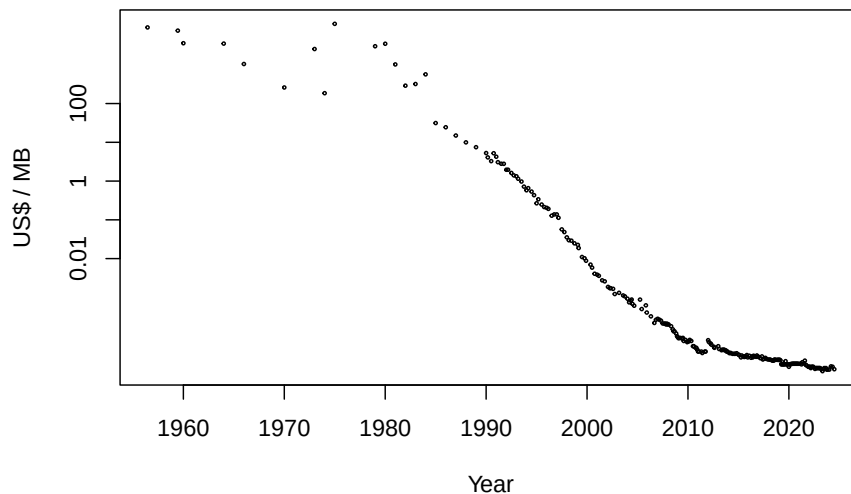


Figure 1.1: The decline in storage costs per megabyte in consumer hard drives.

1.2 Overall Philosophy

1.2.1 Leverage what already exists

Many of the tools that are needed to monitor biodiversity already exist, and the goal of WildlifeSystems is to bring these tools together in a coherent and integrated way. This will allow researchers and others to build on existing technologies to develop new tools and applications that can be used to monitor biodiversity in a wide range of environments. These tools are designed to be flexible, scalable, and easy to use, and to provide a platform for the development of new technologies and applications.

1.2.2 Innovate where necessary

“Technology is stuff that doesn’t work yet.”

— Bran Ferren

Where suitable technological solutions already exist, use them. Limit technological research to what needs to be new. Most of the time it’s OK to use existing technology, and focus development on the biodiversity related problem you’re trying to solve. Older technologies are often better understood and more stable. Some of the technologies WildlifeSystems leverages to innovate in the biodiversity sphere are given in Table 1.1.

Table 1.1: Age of technologies underpinning WildlifeSystems.

Technology	Year Introduced	Age in 2026
Ethernet	1973	53
BASH	1989	37
Linux	1991	35

1.2.3 Plan for heterogeneity

Any network of devices will become heterogeneous given sufficient time. The primary driver of this shift to heterogeneity is continuous technological innovation combined with obsolescence of technical components.

Secondary drivers are changes in the purpose, capabilities, and/or scale of the network over time.

The WildlifeSystems ecosystem is designed to be able to handle this heterogeneity, and to allow for the easy integration of new technologies as they become available.

1.2.4 Modularise

The WildlifeSystems ecosystem is designed to be modular, with each component able to be used independently or in combination with other components. This allows users to build custom solutions that meet their specific needs, and to easily add or remove components as required.

Defined interfaces between components allow for easy integration of new technologies, and for components to be replaced or upgraded without affecting the rest of the system. This also lets contributors to focus on what they are best at.

1.2.5 Open Source

1.3 Structure of Wildlife Systems

1.3.1 Packages

Chapter 2

Nodes

The fundamental unit of the WildlifeSystems platform is the node. A node is a physical device that can be used to collect data from sensors, control devices, and communicate with other nodes. Nodes are (typically) Raspberry Pi computers, packaged with an array of environmental sensors, microphone(s), and/or camera(s).

The WildlifeSystems software is used to configure a node to collect data from the sensors and what to do with the data collected. Typically this will be to send the data for external storage and analysis, but it is also possible to store the data locally on the node. Data can also be processed locally on the node, for example to detect a specific event (e.g. a sound or image) and then send a notification to the user. The WildlifeSystems software is designed to be modular, so that new sensors and devices can be added easily, and the data processing can be configured to suit the user's needs.

2.1 Standard node

There is no such thing as a 'standard node', apart from the core software interfaces. In some deployments, however, there will be a great deal of standardisation. The Urban Research Station deployment at the Natural History Museum, London, for example, has a number of nodes that are identically configured. Although even here, there are differences in the sensors used, and the data processing that is done. The nodes are all configured to send data to the same server (the Data Ecosystem) but the exact configuration of each node is driven by current research needs. In this way a network of WildlifeSystems nodes can be seen as a platform for facilitating research, rather than as a single-purpose device.

To make it easier to deploy and work with individual nodes, it is recommended

that some commonly used sensors are deployed in the standard fashion described below.

2.1.1 Self-monitoring

Electricity and water should not mix. While suitable enclosures go a long way to preventing an unfortunate meeting, many nodes are deployed outside in the elements. The addition of a cheap DHT11 sensor internally to monitor the humidity and temperature of the enclosure may help identify a potentially damaging situation before it becomes problematic. [TODO: See PRTG integration].

This sensor is recommended to be attached to GPIO pin 4.

2.1.2 i2s microphones

TODO

2.1.3 1-Wire bus

GPIO 17

2.2 Enclosures

2.2.1 Fixing inside enclosures

2.2.2 Attaching sensors

TODO: to sockets, not directly

TODO: breakout boards

TODO: custom boards

2.2.3 Power solutions

TODO: PoE

Chapter 3

How nodes work

After installation configuration files are placed in `/etc/ws` that define the operation of the node.

On startup, WildlifeSystems nodes run three systemd services that are responsible for processing audio, images, and environmental sensors. These sensors read the appropriate configuration files, and start collecting and recording data.

3.1 Audio

The service `ws-run-audio` is configured by `/etc/ws/audio.conf` and is responsible for running the audio processing pipeline. The blocks in the pipeline are `record`, `sleep`, `upload-delete`, and `cleanup`.

3.1.1 `init`

3.1.2 `run`

3.1.2.1 `record`

3.1.2.2 `sleep`

3.1.2.3 `upload-delete`

3.1.2.4 `cleanup`

In situations where network connection is temporarily unavailable it is possible that audio files will collect on the advice. These are not automatically processed

during `upload-delete` and their processing must be handled manually, or by creating a processing script in `cleanup`.

Chapter 4

Sensor Networks

Chapter 5

Environmental Sensors

5.1 How sensors work

5.1.1 Temperature

5.1.1.1 Thermistors

A thermistor is a type of resistor whose resistance changes significantly with temperature. The word is a portmanteau of thermal and resistor.

5.1.2 Humidity

5.1.2.1 Capacitive Humidity Sensors

A capacitive humidity sensor consists of a small capacitor with a hygroscopic dielectric material between two electrodes. The dielectric material, usually a plastic or polymer, has a low dielectric constant when dry. In the presence of moisture, the dielectric constant increases due to water vapor, which has a much higher dielectric constant. As moisture is absorbed, the sensor's capacitance increases. The amount of moisture absorbed depends on the surrounding temperature and water vapour pressure, which also affects the hygroscopic dielectric material used in the sensor.

5.1.3 Air Pressure

5.1.4 Gases

5.1.4.1 Heated Gas Resistance

Heated gas sensors are used to detect gases in the air. The sensor consists of a metal oxide semiconductor that is heated to a high temperature. Volatile organic compounds (VOCs) in the air change the resistance of the sensor.

These sensors are typically used indoors and have a short lifespan (perhaps two years). As such they are rarely used in environmental sensing applications, but are included on some common sensors such as the BME680.

Chapter 6

Sensors in WildlifeSystems

The WildlifeSystems platform comes with support for some popular existing environmental sensors, although there are many on the market and the range available is subject to constant change. The modular nature of WildlifeSystems allows for new sensors to be easily integrated if the need arises.

WildlifeSystems takes an abstracted approach to sensors. Interface modules are provided for each sensor, which can be used to read the sensor data into a standard format.

As many physical packages contain multiple sensors (e.g. the supported BME680 can monitor temperature, humidity, air pressure and air quality) there is a conceptual difference between a “device” and the one or more sensors on a device.

6.1 Devices included in the base system

The Raspberry Pi does not come with environmental sensors, however there are several onboard sensors that are used to monitor the operation of the hardware, to prevent crucial components from overheating, including the temperature of the CPU and GPU chips. WildlifeSystems provides access to these sensors through the **sensor-onboard** package, as well as providing some *software sensors* that report the free memory and free SD storage available. These can be useful for detecting and resolving possible issues on a sensor node before a serious problem arises.

6.2 Installing sensor support

Support for sensors is installed as part of the node installation process, however it is possible to install the `sensor-control` abstraction layer onto any Raspberry Pi system using the command below.

```
sudo apt install sensor-control
```

This will install the `sensor-control` and `sensor-onboard` scripts into the system, as well as installing a small number of supporting packages if they are not already installed.

6.3 Reading data from a device

The sensor read command, `sr`, can be used to list the available devices on a given sensor.

```
sr onboard list
```

The sensor read command, `sr`, can also be used to read data from a named sensor. Not specifying a sensor is equivalent to reading all sensors on that device (i.e. using the wildcard `*`).

```
sr onboard
sr onboard *
sr onboard onboard_gpu
```

This will give a JSON string listing information about each sensor (or just the specified sensor), and the current reading. This information can be presented in a more human readable form by piping the output to the program `jq`, a command line JSON processor.

```
sr onboard | jq
```

6.4 The sensor reading process

The sensor reading process in `WildlifeSystems` has five steps.

1. The `sr` command identifies which device script to route the request to.
2. The sensor script calls the `sc-prototype` script to obtain a template (“prototype”) of the JSON request.

3. The sensor script accesses the relevant sensor(s) and populates the values in a template for each sensor reading, before returning a JSON array of populated readings to **sr**.
4. The **sr** script populates additional information for each reading, providing the **node_id** and a **timestamp** for each.
5. **sr** returns the finalised JSON array to the user.

6.5 Installing new sensors

New sensors can be installed using the apt package manager once the Wildlife Systems APT repository has been added to the system. Instructions can be found on the webpage [Configuring APT for Wildlife Systems](#)..

```
sudo apt install sensor-<sensor-name>
```


Chapter 7

Supported sensors

7.1 List of supported sensors

7.1.1 BME680

4-in-1 sensor that provides humidity, temperature, pressure, and gas readings.

7.1.2 DS18B20

The DS18B20 is a digital thermometer that provides Celsius measurements over a 1-Wire interface. It is a low-cost, low-power device that can be used for a variety of environmental monitoring applications. Waterproof versions are available, making it suitable for use in water and soils.

```
sudo apt install sensor-ds18b20  
sudo sr ds18b20
```

7.1.3 DHT11

The DHT11 is a low-cost temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and sends the data by digital signals on the data pin.

The low-cost nature of the sensor makes it useful despite several shortcomings, e.g. for monitoring the temperature and humidity inside electronics enclosures where accuracy is not critical but the sensor can provide critical early warning of overheating or condensation.

```
sudo apt install sensor-dht11  
  
sudo sr dht11
```

7.1.4 LV-MaxSonar-EZ

A series of ultrasonic distance sensors that can measure distances from 2 cm to 4 m with high accuracy. They are commonly used in robotics and automation applications, but can also be used for environmental monitoring.

```
sudo apt install sensor-hrllv  
  
sudo sr hrllv
```

Chapter 8

Implementing new sensors

New sensors should be implemented as new Debian packages, and submitted via GitHub. Packages have a standard format, and should be named `sensor-<sensor-name>` where `<sensor-name>` is short, descriptive of the sensor (e.g. model number), and unique within the WildlifeSystems ecosystem.

As many people implement various sensors on the Raspberry Pi, it is likely that some sort of solution is already available, that can be tweaked to output readings in the standard format required. However, you must ensure that any licensing conditions are met. In particular, an open license is required if submitting your sensor package to WildlifeSystems for inclusion in the ecosystem.

8.1 Reading the sensor

File `sensor-<sensor-name>` provides the functionality to read the prototype JSON, populate the JSON with sensor readings, and print the output.

The file can be written in any programming or scripting language, but to prevent overhead consideration should be given to minimising the number of new packages installed.

8.1.1 In bash

```
#!/bin/bash

# Read the prototype JSON
JSON=$(sc-prototype)
```

```

echo -n "["

# Code to read the sensor value into GPU_TEMP

# Use `jq` to modify JSON
SENSOR=$(echo $JSON | jq ".sensor |= \"onboard_gpu\" | .measures |= \"temperature\" |"
echo -n $SENSOR

echo "]"

```

8.1.2 In Python

```

#!/
import os
import json

# Read the prototype JSON
stream = os.popen('sc-prototype')
output = stream.read()

# Pre-populate with sensor metadata
temperature = json.loads(output)
temperature["sensor"] = "bme680_temperature"
temperature["measures"] = "temperature"
temperature["unit"] = "Celsius"

# Code to read sensor and output in variable `sensor_reading`
temperature["value"] = sensor_reading

# Output the JSON in an array
print("[", json.dumps(temperature), "]")

```

8.2 Submitting packages to WildlifeSystems

Submitting packages (where licensing allows) to WildlifeSystems allows the ecosystem to be developed and sustained collaboratively by the user community.

Packages can be sent to the administrators as a compressed file, or a request can be sent to fork an existing GitHub repository. Contact details can be found at <https://wildlife.systems/contact.html>.

Chapter 9

Sensor Data Tools

The `sensor-tools` package provides command-line utilities for processing, analysing, and visualising sensor data collected by WildlifeSystems nodes. The package includes three main binaries: `sensor-data`, `sensor-mon`, and `sensor-plot`.

9.1 Installation

The `sensor-tools` package can be installed using the `apt` package manager:

```
sudo apt install sensor-tools
```

This will install all three tools and their dependencies.

9.2 `sensor-data`

`sensor-data` is the primary tool for processing and analysing sensor data files. It provides several subcommands for different tasks.

9.2.1 Basic Usage

```
sensor-data <command> [options] [files...]
```

If no input files are specified, data is read from `stdin`. Input files can be JSON (`.out` format with one JSON array per line) or CSV.

9.2.2 Commands

9.2.2.1 transform

Transform sensor data between JSON and CSV formats, with optional filtering.

```
# Convert JSON to CSV
sensor-data transform -of csv sensor1.out

# Convert with error filtering
sensor-data transform --remove-errors -o output.csv sensor1.out

# Process all .out files in a directory recursively
sensor-data transform -r -e .out -o output.csv /var/ws/

# Filter by sensor type
cat sensor1.out | sensor-data transform --only-value sensor:ds18b20

# Filter by date range
sensor-data transform --min-date 2026-01-01 --max-date 2026-01-31 sensor.out

# Apply the --clean shorthand (removes empty values, errors, and duplicates)
sensor-data transform --clean sensor1.out
```

9.2.2.2 list-rejects

List readings that would be filtered out by specified filters. This is useful for inspecting data quality issues.

```
# Show readings that would be removed by --clean
sensor-data list-rejects --clean sensor1.out

# Show error readings
sensor-data list-rejects --remove-errors sensor1.out
```

9.2.2.3 count

Count sensor data readings with optional filtering and grouping.

```
# Count all readings
sensor-data count sensor1.out

# Count excluding errors
sensor-data count --remove-errors -r -e .out /var/ws/
```

```

# Count by sensor type
sensor-data count --by-column sensor_id sensor1.out

# Count by time period
sensor-data count --by-day sensor1.out      # YYYY-MM-DD
sensor-data count --by-week sensor1.out     # YYYY-Www (ISO week)
sensor-data count --by-month sensor1.out    # YYYY-MM
sensor-data count --by-year sensor1.out     # YYYY

# Output as CSV or JSON
sensor-data count -b sensor_id -of csv sensor1.out
sensor-data count -b sensor_id -of json sensor1.out

```

9.2.2.4 distinct

List unique values in a specified column.

```

# List unique sensor names
sensor-data distinct sensor sensor1.out

# List unique sensor IDs with counts (sorted by frequency)
sensor-data distinct sensor_id -c -r -e out /var/ws/

# Output as CSV or JSON
sensor-data distinct sensor -of csv sensor1.out
sensor-data distinct sensor -of json -c sensor1.out

```

9.2.2.5 list-errors

Find and list error readings in sensor data files. This detects DS18B20 sensors reporting values of 85 or -127, which indicate communication errors or disconnection.

```

sensor-data list-errors -r -e .out /var/ws/

```

Example output:

```

sensor1.out:2 type=ds18b20 sensor_id=sensor002 value=85

```

9.2.2.6 summarise-errors

Summarise error occurrences by type and sensor.

```
sensor-data summarise-errors -r -e .out /var/ws/
```

Example output:

```
DS18B20 communication error (value=85): 3 occurrences
DS18B20 disconnected or power-on reset (value=-127): 2 occurrences
Total errors: 5
```

9.2.2.7 stats

Calculate statistics for numeric sensor data. Provides comprehensive analysis including basic statistics, quartiles, outlier detection, and time-based metrics.

```
# Statistics for the 'value' column
sensor-data stats sensor1.out

# Statistics for a specific column
sensor-data stats -c temperature sensor1.out

# Clean statistics (excluding empty values and errors)
sensor-data stats --clean sensor1.out

# Follow mode for live updates
sensor-data stats -f sensor1.out
```

Example output:

Statistics:

Time Range:

```
First:    2026-01-14 22:13:20 (1768529600)
Last:     2026-01-15 01:13:20 (1768540400)
Duration: 3h 0m 0s (10800 seconds)
Rate:     1.33 readings/hour, 32.00 readings/day
```

Sampling:

```
Typical interval: 3600s
Gaps detected:    0
```

value:

```
Count:    100
Min:      18.5
Max:      26.3
Range:    7.8
```



```
Mean:      22.1
StdDev:    1.8
```

```
Quartiles:
  Q1 (25%): 20.5
  Median:   22.0
  Q3 (75%): 23.5
  IQR:      3.0
```

```
Outliers (1.5*IQR):
  Count:    2
  Percent:  2.0%
```

9.2.2.8 latest

Show the latest timestamp for each sensor.

```
# Get latest timestamp per sensor
sensor-data latest sensor1.out

# Limit output to first 5 sensors
sensor-data latest -n 5 sensor1.out

# Output as CSV or JSON
sensor-data latest -of csv sensor1.out
sensor-data latest -of json sensor1.out
```

9.2.3 Common Options

Most commands share these options:

Option	Description
<code>-r, --recursive</code>	Recursively process subdirectories
<code>-e, --extension <ext></code>	Filter files by extension (e.g., <code>.out</code>)
<code>-d, --depth <n></code>	Maximum recursion depth
<code>-if, --input-format <fmt></code>	Input format: <code>json</code> or <code>csv</code>
<code>-of, --output-format <fmt></code>	Output format: <code>json</code> , <code>csv</code> , or <code>human</code>
<code>-o, --output <file></code>	Output file (default: <code>stdout</code>)
<code>-v</code>	Verbose output
<code>-V</code>	Very verbose output

9.2.4 Filtering Options

Option	Description
<code>--min-date <date></code>	Include only readings on or after this date
<code>--max-date <date></code>	Include only readings on or before this date
<code>--remove-errors</code>	Remove DS18B20 error readings (value=85 or -127)
<code>--remove-empty-json</code>	Remove empty JSON input lines
<code>--not-empty <column></code>	Skip rows where column is empty
<code>--not-null <column></code>	Skip rows where column is “null”
<code>--only-value <col:val></code>	Only include rows where column equals value
<code>--exclude-value <col:val></code>	Exclude rows where column equals value
<code>--allowed-values <col> <vals></code>	Only include rows where column is in allowed values
<code>--unique</code>	Remove duplicate rows
<code>--clean</code>	Shorthand for common cleaning options
<code>--tail <n></code>	Only read last n lines from each file

9.2.5 Date Formats

The `--min-date` and `--max-date` options accept:

- ISO 8601: 2026-01-17 or 2026-01-17T14:30:00
- UK format: 17/01/2026
- Unix timestamp: 1768529600

9.3 sensor-mon

`sensor-mon` provides an interactive ncurses-based terminal interface for real-time monitoring of sensor data.

9.3.1 Usage

```
sensor-mon
```

When started, `sensor-mon` scans for executable `sensor-*` programs in the `PATH`, verifies they respond correctly to the “identify” command, and displays a list of available sensors.

9.3.2 Controls

Key	Action
1-9	Select sensor to monitor
b	Return to sensor list
[	Show previous result (multiple readings)
]	Show next result (multiple readings)
q	Quit

9.3.3 Display

The sensor view shows:

- **Top half:** Formatted JSON output from the selected sensor
- **Bottom half:** Real-time graph of the “value” field, updated every second

Colour is used when available:

- Green: Frame and borders
- Blue: Graph data points and lines
- Reversed: Header text background

9.3.4 Sensor Requirements

For a sensor application to be detected by sensor-mon, it must:

1. Be named with the prefix “sensor-” (e.g., sensor-dht22, sensor-ds18b20)
2. Be executable and in the system PATH
3. Exit with status code 60 when called with the “identify” argument
4. Output JSON data when called with the “all” argument

9.4 sensor-plot

`sensor-plot` displays historical sensor data from `.out` files as graphs. Up to 5 sensors can be displayed simultaneously.

9.4.1 Usage

```
sensor-plot --sensor <SENSOR_ID> [--sensor <SENSOR_ID2>] [PATH]
```

If no PATH is specified, defaults to `/var/ws`.

9.4.2 Options

Option	Description
<code>--sensor <ID></code>	Sensor ID to plot (can be specified up to 5 times)
<code>-r, --recursive</code>	Search subdirectories for data files (default)
<code>-R, --no-recursive</code>	Do not search subdirectories
<code>-e, --extension <ext></code>	Only read files with the specified extension
<code>-h, --help</code>	Display help

9.4.3 Controls

Key	Action
Left/Right Arrow	Scroll the time window
<code>h</code>	Hour mode (1 hour view, 1 minute scroll)
<code>d</code>	Day mode (24 hour view, 1 hour scroll)
<code>w</code>	Week mode (7 day view, 1 day scroll)
<code>m</code>	Month mode (30 day view, 1 week scroll)
<code>y</code>	Year mode (365 day view, 1 month scroll)
<code>r</code>	Reload data from disk
<code>q</code>	Quit

9.4.4 Sensor Colours

Each sensor is displayed in a different colour:

- Sensor 1: Cyan
- Sensor 2: Yellow
- Sensor 3: Magenta
- Sensor 4: Red
- Sensor 5: Blue

9.4.5 Examples

```
# Display a single sensor
sensor-plot --sensor ds18b20_abc123

# Display three sensors together
sensor-plot --sensor temp_indoor --sensor temp_outdoor --sensor humidity

# Specify a different data path
sensor-plot --sensor temp_main /path/to/data/
```

9.5 Data Formats

9.5.1 JSON (.out files)

Each line contains a JSON array with one or more sensor readings:

```
[ { "sensor": "ds18b20", "sensor_id": "28-0123456789ab", "value": 22.5, "unit": "Celsius", "timestamp": 1768529600 }
```

9.5.2 CSV

Standard CSV with header row:

```
sensor,sensor_id,value,unit,timestamp
ds18b20,28-0123456789ab,22.5,Celsius,1768529600
```


Chapter 10

Sound Devices

10.1 How sound devices work

Chapter 11

Sound devices in WildlifeSystems

11.1 Installing sound device support

Support for sound devices is installed as part of the node installation process, however it is possible to install the `sound-device-control` abstraction layer onto any Raspberry Pi system using the command below.

```
wget -O - https://github.com/wildlife-systems/sound-device-control/raw/master/install | sudo bash
```

The installation will install the `alsa-utils` package on Ubuntu/Debian systems, and add the current user to the `audio` group.

11.2 Supported sound devices

The WildlifeSystems platform supports several popular sound capture and playback devices.

11.2.1 Raspberry Pi onboard audio

11.2.2 AudioInjector Zero

11.2.3 AudioInjector Octo

11.2.4 AudioInjector Ultra

11.2.5 I2C microphone (e.g. Google Voice Hat)

Limited to 48kHz sample rate, no record volume control.

11.2.6 Respeaker 6 mic array

11.2.7 Audio+ DAC

11.2.8 Wolfson Audio Card

11.3 Installing a sound device

To install support for a sound device, run the following command.

```
sdc-inst <sound-device-name>
```

To get a list of names of supported devices, run the following command.

```
sdc-inst list
```

Chapter 12

Imaging Devices

12.1 How imaging devices work

Chapter 13

Imaging devices in WildlifeSystems

Chapter 14

Power Management

Power management on the Raspberry Pi is useful when deployments are made that are powered by batteries and/or renewable sources such as solar power that are intermittent.

In addition, there are small environmental benefits on consuming less power on systems which have continual grid power.

The `pi-pwr` script can be used to turn off unused functionality, either always or just when it is not required.

14.1 Installation of power management tools

The power management tools are installed as part of the node installation process, however they can be easily installed independently on any Raspberry Pi system.

```
wget -O - https://github.com/wildlife-systems/pi-pwr/raw/master/install | sudo bash
```

14.2 Turning functionality on and off

14.3 Considerations

Disabling all network functionality will prevent the node from communicating until either the functionality is turned back on or the Raspberry Pi is restarted.

If disabling all connectivity is desired periodically then the functionality to turn these systems back on must be scripted.

14.3.1 A note on `sudo`

Raspberry Pi OS (and previously Raspbian) allows the default user to run `sudo` without a password. This is not true for other Linux distributions, such as Ubuntu. This could lead to a password prompt when using `pi-pwr`. As nodes are designed to run autonomously, the installation process for `ws-node` will configure `pi-pwr` to not require a `sudo` password.

Chapter 15

Indicators and heartbeats

The script `ws-indicate` is used to indicate the device's status using the LED(s) on board the Raspberry Pi. The script `ws-heartbeat` can be used to transmit the device's status to a user-defined script that could provide logging, or submit the status to an online dashboard.

15.1 Installation of `ws-indicate` and `ws-heartbeat`

These tools are installed as part of the node installation process.

15.2 Indicators

Internally `ws-indicate` makes repeated calls to `pi-pwr` to control the LED(s). There are three indicator routines, heartbeat (quick flash of LEDs in order to show device is functioning), countdown (flashes power LED), and record (power LED on, action LED off).

```
sudo ws-indicate
sudo ws-indicate countdown 5    # counts down from 5
sudo ws-indicate record action  # record light is on while action is executed
```

15.3 Heartbeat

The script `ws-heartbeat` is used to send a heartbeat signal to the `devices.wildlife.systems` server to indicate that the node is alive and connected, as

well as to provide some information to assist problem diagnosis. The data sent comes from the onboard sensor readings (**sr onboard**) and the server stores the node ID, timestamp, CPU and GPU temperatures, and the amount of memory and storage used. The node must be registered with WildlifeSystems for this data to be stored.

The script may be run for debugging purposes at any time as follows.

```
ws-heartbeat
```

The script will exit silently on success.

Chapter 16

Integration with monitoring tools

16.1 PRTG

As the devices are Raspberry Pi based, we can use the PRTG monitoring tool to monitor standard aspects of their performance including CPU, memory, disk space, and network traffic.

16.1.1 Monitoring custom sensors

Some sensors attached to WildlifeSystems nodes may be used to monitor aspects of the devices themselves, rather than the environment. Using custom PRTG scripts it is possible to include these measurements in PRTG reports and alerts.

The following example reports the humidity of the device enclosure measured by a DHT11 sensor. It makes use of the WildlifeSystems `sr` command to read the sensor in a standard JSON format, and converts the reading to a format expected by PRTG.

```
#!/bin/bash

HV=$(sudo sr dht11 | jq 'map(select(.sensor == "dht11_humidity")) | .[0].value')

if (( $HV < 75 )); then
  /bin/echo "0:$HV:Acceptable humidity"
else
  /bin/echo "4:$HV:Too humid"
fi
```


Chapter 17

Developer Guidelines

17.1 Documentation

There are three main kinds of documentation in the WildlifeSystems project.

1. **Code comments:** Primarily for future you (or someone similar) to get to grips with your code. *Why does this code do this?*
2. **Package documentation:** Describes what your package does, how it interacts with the larger system. *What will this package do for me?*
3. **Overall project documentation:** This manual. *How do I use the entire system for my research?*

17.2 File system layout

- /etc/ws	# Configuration files (editable by end user)
- /usr/bin	# Main scripts (do not edit)
- /var/ws	# Data storage (do not edit)
- audio	# Audio data
- bluetooth	# Bluetooth data

Chapter 18

Server Tools

These tools are only needed for running and maintaining the wildlife.systems webserver.

18.1 Installation

```
git clone git@github.com:Wildlife-Systems/wildlife.systems-tools.git
cd wildlife.systems-tools
./install
cp .ws-db.php ~/
#Edit ~/.ws-db.php to connect to the wildlife-systems database
```

18.2 Adding nodes and receiving a token

```
ws-node-add <node_id>
```

Adding a node by it's ID (serial number of the Raspberry Pi) will add the device to the wildlife.systems database, generate a token, and display the token (a UUID) in the console. This token is used to authenticate the device when it sends data to wildlife.systems.

18.3 Removing a node

```
ws-node-remove <node_id>
```


Appendix A

A note on sudo

Raspberry Pi OS (and previously Raspbian) allows the default user to run `sudo` without a password. This is not true for other Linux distributions, such as Ubuntu. This could lead to a password prompt when using scripts such as `sr` and `ws-indicate`.

When using a standard node configuration this is not an issue as the `systemd` services that launch these scripts are configured to run as root. However, if you are using these scripts manually you may need to run them with `sudo` (e.g. `sudo sr onboard`) to get expected behaviour.

Appendix B

Return codes

The various scripts that form the WildlifeSystems ecosystem use a standard set of return codes.

B.1 00-09 Script functionality

Code	Label	Description
0	OK	Terminated normally. No error.
1	Already running	The script determined it was already running and terminated.
2	No arguments	The script requires arguments but none given. Will give help text.
3	Config file error	The config file is missing or malformed.

B.2 10-19 Parameter problems

Code	Label	Description
10	Invalid argument	One or more of the arguments to the script was invalid.
11	Incorrect filename pattern	A standard filename pattern was expected. Allowed values are 'timestamp'.

B.3 20-29 Sensor problems

Code	Label	Description
20	Unknown device	This device is not supported, or software is not installed.
21	Unknown sensor	This sensor is not known on this device.

B.4 30-39 Sound device problems

Code	Label	Description
30	Unsupported device	The specified device is unsupported.
31	Unsupported feature	This feature is not present on the sound device.
32	Unknown feature	The feature requested is unknown.

B.5 40-49 Image device problems

Code	Label	Description
40	Unsupported device	The specified device is unsupported.
41	Unsupported feature	This feature is not present on the sound device.
42	Unknown feature	The feature requested is unknown.

B.6 50-59 Power management problems

Code	Label	Description
50	Disallowed value (should be on or off)	The specified device is unsupported.
51	Disallowed value (should be on, off or default)	This feature is not present on the sound device.

B.7 60-69 Special meanings

Code	Label	Description
60	Identify as a sensor device script	Returned by sensor device script in response to first param